ATS App Summary

Overview of the AASTHIX Talent application, its user surface, core workflows, and supporting

Dashboard pages	**32**	**API routes**	**85**
Lib modules	**108**	**SQL migrations**	**44**

What It Is

AASTHIX Talent is a full-stack applicant tracking system built on Next.js, React, Tailwind, and PostgreSQL. The repo combines recruiting operations, careers intake, chat, analytics, screening, and multiple matching pipelines inside one workspace.

Who It's For

Primary users are internal recruiting teams: recruiters, admins, hiring managers, and coordinators who manage jobs, candidates, applications, interviews, approvals, and team collaboration.

At A Glance

- Dashboard navigation covers candidates, jobs, requisitions, pipeline, clients, interviews, chat, analytics, alerts, screening, audit, roadmap, recruiter copilot, and recruiter workload.
- Public careers routes let external candidates browse open roles and submit applications with resumes.
- RBAC and team-based visibility are implemented through permission keys plus job-team membership.
- The repo includes both deterministic and AI-assisted candidate-job matching paths.

Pure AI matching leads to higher token usage.
The hybrid system optimizes this by performing initial candidate scoring via a rule-based Python engine, and then applying AI reranking on a smaller subset.

Key Capabilities

Representative features visible in routes, components, APIs, and docs.

Hiring operations	Jobs, requisitions, candidate records, applications, pipeline board, shortlist and status flows.
Interviews and screening	Interview scheduling/alerts plus screening tests, analytics, resends, and time-to-submit tracking.
Recruiter productivity	Recruiter copilot, workload views, activity center, alerts, dashboard metrics, and audit surfaces.
Team collaboration	Built-in chat conversations/messages, admin invites, permissions, and job-team visibility rules.
Careers portal	Public jobs listing, JD modal, application form, resume upload, confirmation email, and funnel tracking events.
Matching stack	Rule-based Python matcher, optional OpenAI/vector matching, async queue processing, and run-status polling.

Notable Repo-Backed Workflows

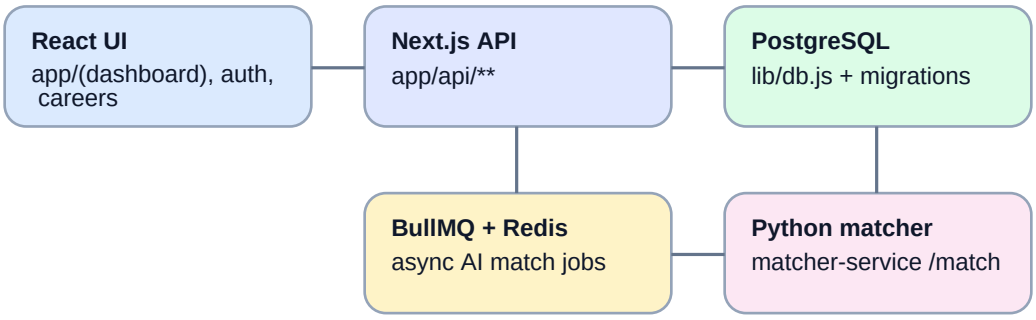
- Candidate intake: bulk import, resume parsing, public careers apply flow, and candidate CRUD APIs.
- Hiring execution: jobs and applications feed dashboard cards, pipeline stages, interview alerts, and analytics endpoints.
- Governance: permission keys, admin pages, audit feeds, and owner-or-team visibility filters shape access.
- Matching: no-AI matching can prefilter in Postgres and then call the Python /match service; async AI matching can enqueue BullMQ jobs and process them in a worker.

Evidence Sources Used

- `package.json` for runtime stack and scripts.
- `README.md` and `.env.example` for baseline setup and env requirements.
- `app/(dashboard)`, `app/(auth)`, `app/careers`, and `app/api/**` for pages and APIs.
- `lib/` modules for auth, RBAC, queueing, matching, and careers workflows.
- `docs/no-ai-matching.md`, `docs/PERMISSION_MATRIX.md`, and `matcher-service/README.md` for supporting implementation detail.

Architecture And Runbook

Compact view of components, services, and minimal startup steps.



How It Works

- React pages render the dashboard, auth screens, and careers portal; client code fetches JSON from route handlers under `app/api/**`.
- Middleware protects non-public pages by checking for the auth cookie, while APIs enforce JWT-backed identity and permission checks through `lib/auth.ts`, `lib/authServer.ts`, and `lib/rbac.ts`.
- PostgreSQL is the system of record. `lib/db.js` creates the pool, and `migrations/*.sql` define the evolving schema for ATS entities and match runs.
- Optional async AI matching uses BullMQ with Redis plus a dedicated worker process (`npm run worker:ai-match`).
- Optional deterministic matching uses the separate `matcher-service` Python server at `/match`.

Minimal Getting Started

- Run `npm install`.
- Create `.env.local` from `.env.example` and set at least `DATABASE_URL` and `JWT_SECRET`; add SMTP, OpenAI, and other values only for the features you plan to use.
- Provision PostgreSQL and apply the SQL files in `migrations/`. A migration runner command was not found in the repo, so this appears to be a manual step.
- Start the app with `npm run dev` and open `http://localhost:3000`.
- Optional: run `python -m matcher_service.simple_server` inside `matcher-service/` for no-AI matching.
- Optional: start Redis and run `npm run worker:ai-match` for queued AI match extraction jobs.

Bottom line

This repo is not just a basic ATS scaffold anymore. The evidence shows a broader recruiting operations platform with public career intake, internal collaboration, admin governance, analytics, screening, and multiple matching paths layered on top of a PostgreSQL-backed Next.js application.